UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información

Centro Universitario Villanueva

Proyecto Final: Desarrollo Web — Jornada Sábado



La Canasta

Sistema de Gestión del Torneo Navideño de Básquetbol

Manual Técnico de Publicación y Uso del Framework

Framework	Yii 2 (versión 2.0.55)
Lenguaje	PHP 8
Base de datos	MySQL / MariaDB
Hosting	InfinityFree (plan gratuito)
URL publicada	https://lacanastayii.gt.tc/

Integrantes del Grupo 1

Nombre	Carné
Ricardo Andree Arias Conde	5190-19-3821
Angel Adrian Arias Conde	5190-18-16246
Darlyng Yasmin Marroquín Sosa	5190-17-24336

Guatemala, octubre de 2026

Índice de contenido

1. Introducción
   1.1 Objetivo del proyecto · 1.2 Framework asignado · 1.3 Alcance funcional · 1.4 URL publicada
2. Manual Técnico de Publicación
   2.1 Prerrequisitos · 2.2 Fase 1: desarrollo y pruebas locales · 2.3 Fase 2: publicación en producción (InfinityFree) · 2.4 Verificación · 2.5 Actualización del sitio
3. Pasos utilizados para hacer uso del framework
   3.1 al 3.10
4. Recomendaciones para el uso del framework
5. Comentarios personales del grupo
6. Anexos
   A. Estructura del proyecto · B. Rutas principales · C. Datos iniciales · D. Requerimientos vs. cumplimiento · E. Repositorio

1. Introducción

1.1 Objetivo del proyecto

El presente proyecto consiste en el desarrollo de una aplicación web usando un **framework de desarrollo para el lenguaje PHP**. El caso asignado es la tienda de artículos deportivos **La Canasta**, que necesita un sistema para registrar los resultados de todo el **Torneo Navideño de Básquetbol**, incluyendo los equipos, sus integrantes y las anotaciones de cada jornada, además de generar reportes estadísticos.

1.2 Framework asignado

El framework asignado al grupo fue **Yii 2**, en su versión reciente **2.0.55**. Se trabajó sobre la plantilla *Yii 2 Basic Project Template*, con **PHP 8** y **MySQL/MariaDB**. La interfaz usa **Bootstrap 5** a través del paquete `yiisoft/yii2-bootstrap5`.

1.3 Alcance funcional

El sistema cumple los siguientes requerimientos del enunciado:

Requerimiento	Estado
Registro de jugadores: nombres y apellidos en campos distintos	**Cumplido**
Fecha de nacimiento y fotografía del jugador	**Cumplido**
Equipo como catálogo, con logo del equipo	**Cumplido**
Registro de puntos por jugador en cada jornada	**Cumplido**
Tipo de tiro como catálogo (libre, dos puntos, tres puntos)	**Cumplido**
Reporte: lista general de jugadores agrupada por equipo (con fotografía)	**Cumplido**
Reporte: jugadores por equipo, ordenados alfabéticamente por apellidos	**Cumplido**
Reporte: máximos anotadores, ordenado de mayor a menor	**Cumplido**
Consulta en pantalla de la ficha completa de un jugador	**Cumplido**

1.4 URL publicada

La aplicación se encuentra publicada y accesible en Internet con certificado SSL (HTTPS):

```
https://lacanastayii.gt.tc/
```

El código fuente completo está disponible en el repositorio del proyecto (ver Anexo E).

2. Manual Técnico de Publicación

Esta sección describe el procedimiento completo, en orden cronológico, para dejar la aplicación funcionando y publicada en Internet. El proceso se organiza en **dos fases claramente diferenciadas**:

1. **Fase 1 — Desarrollo y pruebas locales:** se instala el framework y las dependencias con **Composer**, se crea la base de datos y se prueba todo en una máquina local con **XAMPP**.
2. **Fase 2 — Publicación en producción:** una vez validada localmente, la aplicación se publica en un hosting gratuito (**InfinityFree**).

> **Importante:** Composer, el script `yii` y las migraciones son herramientas del **entorno local de desarrollo**. El servidor de producción **no** las tiene ni las necesita.

2.1 Prerrequisitos

Software necesario, indicando el entorno en el que se utiliza cada herramienta:

Software	Versión recomendada	Entorno	Uso
PHP	8.0 o superior	Local y producción	Lenguaje de la aplicación
Composer	2.x	Local	Gestor de dependencias (instala Yii 2 y librerías)
MySQL / MariaDB	8.0 / 10.4 o superior	Local y producción	Base de datos del sistema
XAMPP	Reciente	Local	Paquete con Apache + MySQL + PHP (entorno de desarrollo)
Git	Reciente	Local	Clonar el repositorio
Navegador web	Chrome / Edge recientes	Local y producción	Acceso a la aplicación
Cliente FTP	FileZilla	Producción	Subir archivos al hosting

> **Extensiones de PHP requeridas:** `pdo_mysql` y `mbstring`. En XAMPP vienen habilitadas por defecto. Se pueden verificar con `php -m`.

2.2 Fase 1 — Desarrollo y pruebas locales (XAMPP)

En esta fase se instala el framework con **Composer** y se prueba la aplicación en la máquina local. **Todos los pasos siguientes son locales**; el servidor de producción no los ejecuta.

2.2.1 Clonar el repositorio

```
git clone https://github.com/ravenc1/canasta_yii.git
cd canasta_yii
```

2.2.2 Instalar dependencias

Instala exactamente las versiones registradas en `composer.lock` (Yii 2, Bootstrap 5, JQuery UI, SwiftMailer, Gii, Debug):

```
composer install
```

2.2.3 Crear la base de datos

Crea la base de datos `lacanasta_db` con codificación `utf8mb4`:

```
mysql -u root -e "CREATE DATABASE IF NOT EXISTS lacanasta_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
```

2.2.4 Ejecutar migraciones (crear tablas + datos iniciales)

```
php yii migrate --interactive=0
```

Esto crea las tablas `equipo`, `jugador`, `tipo_tiro` y `puntuacion`, e inserta los datos iniciales: **6 equipos, 30 jugadores, 3 tipos de tiro y 77 puntuaciones**.

2.2.5 Levantar el servidor de desarrollo

```
php yii serve
```

La aplicación queda disponible en `http://localhost:8080/`.

2.2.6 Pruebas funcionales locales

Antes de publicar, la aplicación se probó de forma manual y funcional en el entorno local. Los 45 casos de prueba (módulos de equipos, jugadores, puntuaciones, reportes, interfaz e integridad de IDs) se documentan en el archivo `PRUEBAS.md` del repositorio. Solo tras validar el funcionamiento local se procedió a la publicación.

2.3 Fase 2 — Publicación en producción (InfinityFree)

Una vez validada la aplicación en local, se publica en **InfinityFree**, un hosting gratuito con **PHP 8.4, MySQL 8.0 / MariaDB 11.4, soporte total de `.htaccess`** y certificados SSL gratuitos. Este servidor tiene dos restricciones importantes que condicionan el despliegue:

- **No hay acceso por SSH ni terminal:** no se puede ejecutar `composer` ni `php yii migrate` en el servidor.
- **El directorio raíz del sitio (*document root*) es `htdocs`** y no puede cambiarse: siempre apunta a la carpeta `htdocs`.

Por eso, en el servidor se resuelve así: la carpeta `vendor` se sube ya construida, los datos se importan por phpMyAdmin y un archivo `.htaccess` en la raíz redirige todo hacia `web/` (el directorio público real de Yii).

2.3.1 Crear la cuenta y el dominio

1. Registrarse en `https://dash.infinityfree.com/register` (solo correo, sin tarjeta).
2. Crear una cuenta de hosting y elegir un subdominio gratuito (en este proyecto, `lacanastayii.gt.tc`).

3. Anotar los datos de acceso FTP que muestra el panel de control.

2.3.2 Crear la base de datos

1. En el panel de control, abrir **MySQL Databases** y crear una base de datos.
2. Anotar los cuatro valores de conexión: **host** (por ejemplo `sqlXXX.infinityfree.com`), **nombre de la base**, **usuario** y **contraseña**. En InfinityFree estos suelen tener el prefijo `if0_`.

2.3.3 Importar los datos (dump SQL)

Como el servidor no permite ejecutar migraciones, se genera un respaldo `.sql` de la base local y se importa:

```
# En la máquina local, generar el respaldo:
mysqldump -u root --no-create-db --default-character-set=utf8mb4 lacanasta_db >
lacanasta_db.sql
```

1. Abrir **phpMyAdmin** desde el panel de InfinityFree.
2. Seleccionar la base de datos creada y entrar a la pestaña **Import**.
3. Subir `lacanasta_db.sql` y confirmar. Deben quedar **6 equipos, 30 jugadores, 3 tipos de tiro y 77 puntuaciones**.

2.3.4 Configurar la conexión

Editar `config/db.php` con los datos obtenidos en el paso 2.3.2:

```
return [
    'class'    => 'yii\db\Connection',
    'dsn'      =>
'mysql:host=sqlXXX.infinityfree.com;dbname=if0_XXXXXXX_lacanasta;charset=utf8mb4',
    'username' => 'if0_XXXXXXX',
    'password' => 'TU_PASSWORD',
    'charset'  => 'utf8mb4',
];
```

> **Recomendado en producción:** en `web/index.php` desactivar el modo de depuración, para no exponer rutas ni trazas de error a los visitantes:
> `YII_DEBUG = false` y `YII_ENV = 'prod'`.

2.3.5 Subir los archivos al servidor

Subir **el contenido del proyecto dentro de `htdocs/`** (es decir, `htdocs/` debe contener `config/`, `controllers/`, `models/`, `views/`, `vendor/`, `runtime/`, `web/` y el `.htaccess` de la raíz). Se recomienda **FTP (FileZilla)** por la cantidad de archivos de `vendor/`.

Elemento	¿Por qué?
`vendor/`	No hay Composer en el servidor; debe subirse ya construida
`runtime/`	Carpeta de trabajo de Yii (caché y logs); debe existir y ser escribible
`web/assets/`	Yii publica aquí los recursos (CSS/JS); debe existir y ser escribible

Elemento	¿Por qué?
`.htaccess` (raíz)	Redirige las peticiones hacia `web/`
`web/.htaccess`	Habilita las URLs amigables (pretty URLs)

2.3.6 Estructura en el servidor y .htaccess

El archivo `.htaccess` de la raíz contiene:

```
RewriteEngine on
RewriteRule ^(.*)$ web/$1 [L]
```

De esta forma, aunque el *document root* apunte a `htdocs`, el contenido público real (el que Yii espera) es el de la carpeta `web/`, y todas las peticiones se enrutan correctamente.

2.3.7 SSL / HTTPS gratuito

Desde el panel de control, usar la herramienta **Free SSL Certificates** para emitir el certificado del dominio. Con esto la aplicación queda accesible por `https://`.

2.4 Verificación de la publicación

Comprobaciones realizadas sobre el sitio en línea:

Comprobación	Resultado
Página de inicio responde y renderiza	OK
HTTPS / SSL activo	OK
Hojas de estilo (CSS) y assets publicados	OK
Imágenes (logos, fotos, favicon) visibles	OK
Módulos de equipos, jugadores y puntuaciones con datos	OK
Reportes (general, por equipo, máximos anotadores y perfil)	OK

2.5 Actualización del sitio

1. Modificar los archivos localmente y probar.
2. Subir por FTP solo los archivos modificados, respetando la estructura de `htdocs/`.
3. Si se cambian migraciones o modelos, actualizar también la base de datos remota por phpMyAdmin.

3. Pasos utilizados para hacer uso del framework

Esta sección describe, en orden, cómo se utilizó Yii 2 para construir la aplicación.

3.1 Creación del proyecto base

Se creó la estructura inicial con **Yii 2 Basic Project Template** mediante Composer. La plantilla genera la organización MVC típica de Yii: `config/`, `controllers/`, `models/`, `views/`, `web/` y un script de consola `yii`.

3.2 Configuración de la aplicación

En `config/web.php` se configuraron los componentes de la aplicación:

- **Base de datos** (`db`) apuntando a `config/db.php`.
- **URL Manager** con `enablePrettyUrl = true` y `showScriptName = false`.
- **Idioma** de la aplicación en español.
- **AssetManager** para la publicación de recursos.

3.3 Migraciones (esquema de base de datos)

En lugar de crear las tablas a mano, se usaron **migraciones**, que versionan el esquema y permiten reconstruir la base de datos con un comando. Se crearon migraciones para las tablas `equipo`, `jugador`, `tipo_tiro` y `puntuacion`, con **claves foráneas** `ON DELETE CASCADE`, y una migración final (*seeder*) con los datos iniciales.

3.4 Modelos con ActiveRecord

Cada tabla se mapeó a un modelo ActiveRecord: `Equipo`, `Jugador`, `TipoTiro` y `Puntuacion`. En los modelos se definieron:

- **Reglas de validación** (`rules()`) y etiquetas de atributos.
- **Relaciones** entre tablas (`hasOne` / `hasMany`), por ejemplo jugador–equipo y puntuación–tipo de tiro.
- Métodos propios, como la carga de archivos (logo y fotografía) y el cálculo automático de puntos según el tipo de tiro.

3.5 Controladores y acciones CRUD

Se implementaron controladores (`EquipoController`, `JugadorController`, `PuntuacionController`, `ReportesController`) con acciones para listar, crear, ver, editar y eliminar. La carga de archivos se maneja con `UploadedFile` y la validación se refuerza con `addError()` cuando un archivo no puede guardarse.

3.6 Vistas y layout

Se desarrollaron las vistas CRUD (listado con `GridView`, formularios reutilizables `_form.php`, detalle) y un **layout principal** (`views/layouts/main.php`) con barra de navegación, menús desplegables y personalización visual (fondo aleatorio, contenedores semitransparentes y favicon dinámico).

3.7 Generador de código Gii

Se usó la herramienta **Gii** de Yii 2 para generar los modelos y el esqueleto de los CRUD a partir de las tablas, y luego se adaptaron a las necesidades del torneo.

3.8 AssetBundle y Bootstrap 5

Se definió un **AssetBundle** (`assets/AppAsset.php`) que registra la hoja de estilos propia y depende de Bootstrap 5, permitiendo una interfaz moderna y responsiva.

3.9 Carga de archivos (logo y fotografía)

Los modelos `Equipo` y `Jugador` incluyen reglas de tipo `file` (extensiones y tamaño máximo de 2 MB) y métodos que guardan los archivos en `web/uploads/`. Un *helper* (`ImagenHelper`) construye las URL y usa una imagen de reserva si el archivo no existe.

3.10 Reportes y consultas

Los reportes combinan el uso de **ActiveRecord** con consultas mediante `Query` y agregaciones SQL. Por ejemplo, los máximos anotadores usan `SUM(puntuacion.puntos)` agrupado por jugador y ordenado de mayor a menor.

4. Recomendaciones para el uso del framework

- **Usar migraciones, no SQL manual.** Versionar el esquema permite reconstruir la base de datos en cualquier equipo con un solo comando y mantener el control de cambios.
- **Dejar que Gii acelere el arranque.** Genera modelos y CRUD base muy rápido; después solo hay que adaptar reglas, vistas y estilos.
- **Configurar `prettyUrl` desde el inicio** y añadir el `.htaccess` correspondiente; las URLs limpias mejoran la presentación del proyecto.
- **Gestionar los entornos.** Mantener `YII_DEBUG` en `false` y `YII_ENV` en `prod` en el servidor para no exponer información interna.
- **Separar la configuración sensible.** Las credenciales de la base de datos deben ir en `config/db.php` (o en archivos locales ignorados por el control de versiones), nunca en el código de la lógica de negocio.
- **Usar las validaciones del modelo.** Concentrar las reglas en `rules()` evita duplicar comprobaciones en los controladores.
- **Preparar el despliegue en hosting compartido.** Si el servidor no tiene Composer ni terminal, subir `vendor/` ya construida e importar la base por phpMyAdmin.
- **Cuidar la seguridad de las subidas.** Restringir extensiones y tamaño máximo en los archivos cargados, y validar siempre en el servidor.
- **Leer la guía oficial.** La documentación de Yii 2 (*yiiframework.com/doc/guide/2.0*) y la comunidad resuelven la mayoría de dudas; conviene revisarla antes de implementar desde cero.

5. Comentarios personales del grupo

Valoración del grupo sobre la facilidad o dificultad de uso del framework Yii 2:

Facilidades encontradas

- La generación automática de código con **Gii** ahorró mucho tiempo en los CRUD.
- **ActiveRecord** simplificó las consultas y las relaciones entre tablas.
- La integración con **Bootstrap 5** permitió una interfaz moderna con poco esfuerzo.
- Las **migraciones** facilitaron trabajar en equipo con una base de datos reproducible.
- La documentación oficial es amplia y está bien estructurada.

Dificultades encontradas

- La **curva de aprendizaje inicial** del patrón MVC y de la configuración de Yii requiere tiempo.
- El despliegue en un hosting gratuito presentó retos por la **falta de terminal y Composer** en el servidor, y por no poder cambiar el *document root*.
- La **carga de archivos** y su validación exige atención especial en los detalles.
- Adaptar el **estilo visual** al gusto del grupo llevó varias iteraciones.

Conclusión del grupo

Yii 2 resultó un framework **robusto y productivo**. Tras superar la curva de aprendizaje inicial, sus herramientas (Gii, ActiveRecord, migraciones y la integración con Bootstrap) permitieron construir la

aplicación de forma ordenada y rápida. La experiencia fue positiva y recomendamos su uso para proyectos web en PHP de tamaño pequeño y mediano.

6. Anexos

Anexo A. Estructura del proyecto

```
canasta_yii/
├── assets/          Definición de assets (AppAsset)
├── commands/        Comandos de consola
├── config/          Configuración (web.php, db.php, params.php, console.php)
├── controllers/     Controladores MVC (Equipo, Jugador, Puntuacion, Reportes, Site)
├── helpers/         Clases auxiliares (ImagenHelper)
├── migrations/      Migraciones y datos iniciales
├── models/          Modelos de datos y formularios
├── views/           Vistas y layout principal
├── web/             Document root (index.php, css, assets, uploads)
├── composer.json    Dependencias del proyecto
├── composer.lock    Versiones exactas de dependencias
├── yii              Script de consola de Yii
└── README.md        Documentación del proyecto
```

Anexo B. Rutas principales

Módulo	Rutas
Equipos	`/equipo/index`, `/create`, `/view`, `/update`, `/delete`
Jugadores	`/jugador/index`, `/create`, `/view`, `/update`, `/delete`
Puntuaciones	`/puntuacion/index`, `/create`, `/delete`
Reportes	`/reportes/general`, `/por-equipo`, `/maximos-anotadores`, `/perfil?id=N`

Anexo C. Datos iniciales (migración semilla)

Entidad	Cantidad
Equipos	6
Jugadores	30
Tipos de tiro	3 (Libre, Dos puntos, Tres puntos)
Puntuaciones	77

Anexo D. Requerimientos vs. cumplimiento

Requerimiento del enunciado	Evidencia
Nombres y apellidos en campos diferentes	Tabla `jugador`; formulario de alta
Fecha de nacimiento	Campo `fecha_nacimiento` (tipo fecha)
Fotografía (imagen)	Carga de archivo con validación

Requerimiento del enunciado	Evidencia
Equipo con logo (catálogo)	CRUD de equipos + carga de logo
Jornada, puntos y tipo de tiro	Tabla `puntuacion` + catálogo `tipo_tiro`
Lista general de jugadores agrupada por equipo	Reporte `/reportes/general`
Jugadores por equipo, orden por apellidos	Reporte `/reportes/por-equipo`
Máximos anotadores, de mayor a menor	Reporte `/reportes/maximos-anotadores`
Ficha completa de un jugador	Reporte `/reportes/perfil`

Anexo E. Repositorio del código fuente

```
https://github.com/ravenc1/canasta_yii
```

El repositorio incluye el código fuente completo, las migraciones y las instrucciones de instalación y publicación (`README.md`).